

Lec26

Express.js

Express.js is a lightweight and fast web framework for Node.js.

It helps us build:

- Web servers
- APIs
- Backend applications
- Dynamic websites

Officially, Express.js is a framework built on top of Node.js.

Why do we need Express.js?

Without Express:

In Node.js, server creation looks like this:

```
const http = require('http');
```

```
http.createServer((req, res) => {  
  if(req.url === '/') {  
    res.end("Home");  
  }  
}).listen(3000);
```

Managing multiple routes becomes difficult.

With Express:

```
const express = require('express');  
const app = express();
```

```
app.get('/', (req, res) => {  
  res.send("Home Page");  
});
```

```
app.listen(3000);
```

Cleaner and easier.

Features of Express.js

- Fast
- Lightweight
- Easy routing
- Middleware support
- API development
- Easy request handling
- Easy response handling

Installation

```
npm init -y
```

```
npm install express
```

Package gets added in:

```
package.json
```

Server Creation in Express.js

Basic Server Creation

```
const express = require('express');  
const app = express();  
  
app.listen(3000, () => {  
  console.log("Server running on port 3000");  
});
```

Routes in Express.js

Routing means deciding what response to send for a specific URL.

Example:

- / → Home
- /login → Login Page
- /about → About Page

Multiple Routes

```
app.get('/', (req, res) => {  
  res.send("Home");  
});
```

```
app.get('/about', (req, res) => {  
  res.send("About");  
});
```

```
app.get('/contact', (req, res) => {  
  res.send("Contact");  
});
```

Route Parameters

Example:

/user/101

Route:

```
app.get('/user/:id', (req, res) => {  
  res.send(req.params.id);  
});
```

Output:

101

Requests in Express.js

Request means data sent by client to server.

Express provides:

req

object.

Important Request Properties

- `req.url`
- `req.method`
- `req.params`
- `req.query`
- `req.body`

req.params

Used for route parameters.

Example:

```
app.get('/student/:id', (req, res) => {  
  console.log(req.params);  
  res.send(req.params.id);  
});
```

URL:

/student/25

Output:

25

req.query

Used for query parameters.

Example URL:

/products?category=mobile

Route:

```
app.get('/products', (req, res) => {  
  res.send(req.query.category);  
});
```

Output:

mobile

req.body

Used for POST data.

Example:

Form submits:

- Name
- Email

Need middleware to parse body.

```
app.use(express.json());
```

Example:

```
app.post('/submit', (req, res) => {  
  console.log(req.body);  
  res.send("Received");  
});
```

Responses in Express.js

Response means data sent from server to client.

Express provides:

`res`

object.

Common Response Methods

`res.send()`

`res.json()`

`res.sendFile()`

`res.status()`

res.send()

Sends text/HTML.

```
res.send("Hello Students");
```

res.json()

Sends JSON.

```
res.json({  
  name: "Naman",  
  age: 22  
});
```

Output:

```
{  
  "name": "Naman",  
  "age": 22  
}
```

res.status()

Sets HTTP status.

Example:

```
res.status(404).send("Not Found");
```

res.sendFile()

Send HTML file.

```
res.sendFile(__dirname + '/index.html');
```

Middleware in Express.js

Middleware = Function that runs between:

- Request received
- Response sent

Flow:

Client → Middleware → Route → Response

Syntax

```
app.use((req, res, next) => {  
  console.log("Middleware Executed");  
  next();  
});
```

Middleware can:

- Modify request
- Modify response
- End request-response cycle
- Call next middleware

next()

Moves control to next middleware or route.

```
next();
```

Very important.

Without next:

Request gets stuck.

Example

```
app.use((req, res, next) => {  
  console.log("Request received");  
  next();  
});
```

```
app.get('/', (req, res) => {  
  res.send("Home");  
});
```

Types of Middleware

Application Middleware

`app.use(...)`

Runs for all routes.

Route Middleware

Runs only on specific routes.

```
const auth = (req, res, next) => {  
  console.log("Checking user");  
  next();  
};
```

```
app.get('/dashboard', auth, (req, res) => {  
  res.send("Dashboard");  
});
```

Built-in Middleware

Express provides:

`express.json()`

`express.static()`

Example:

```
app.use(express.json());
```

Middleware Example: Authentication

```
const auth = (req, res, next) => {  
  const isLoggedIn = true;  
  if (isLoggedIn) {  
    next();  
  } else {  
    res.send("Unauthorized");  
  }  
};
```

Use:

```
app.get('/profile', auth, (req, res) => {  
  res.send("Profile Page");  
});
```

Full Example

```
const express = require('express');  
  
const app = express();  
  
app.use(express.json());  
  
app.use((req, res, next) => {  
  
  console.log("Middleware Running");  
  
  next();  
  
});
```

```
app.get('/', (req, res) => {  
  res.send("Home Page");  
});  
  
app.get('/about', (req, res) => {  
  res.send("About Page");  
});  
  
app.post('/data', (req, res) => {  
  res.json(req.body);  
});  
  
app.listen(3000, () => {  
  console.log("Server running");  
});
```

Task

Create a basic Express server with routes and middleware